

3_Parallelism

- **Doing calculations simultaneously**
- Concurrency is not the same as parallelism
 - Concurrency means multiple processes at the same time, doesn't mean they are necessarily acting simultaneously

Strategies for achieving parallelism

- **Pipelining - is an instance of superscalar processors** (making more than 1 instruction make progress at one cycle) (technically superscalar processors can exist without pipelining as well but this is not covered in the course)
- **Vectorization** - do operations on multiple items at the same time
- **Multicore** - multiple core processors
- **Heterogenous computing (with GPU)** - GPUs are faster at performing certain operations, so we can use GPUs for those and then the CPU for those operations where GPUs aren't faster

Types of parallelism

We can categorize different parallelization strategies in the following categories:

- **ILP - Instruction Level Parallelism**
- **TLP - Thread Level Parallelism**
- **DLP - Data Level Parallelism**

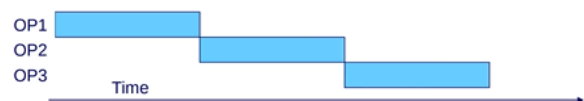
	Horizontal	Vertical
ILP	Superscalar / VLIW	Pipelined
TLP	Multi-core SMT	Interleaved / switch-on-event multithreading
DLP	SIMD / SIMT	Vector / temporal SIMT

Pipelining

- Reorganise tasks to do tasks simultaneously where possible
 - For example, if you're washing and drying clothes, after the first wash finishes you can put the clothes in the dryer which frees the washing machine so you can do the next wash while the first batch of clothes is drying
- Doesn't help with the individual length of the operations, but with the overall length of the whole process
- The amount of speedup possible is limited by the slowest operation in the pipeline

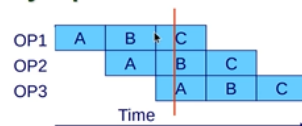
Visual example

Unpipelined



- Cannot start new operation until previous one completes

3-Way Pipelined



- Up to 3 operations in process simultaneously

- To efficiently apply pipelining you need to be able to split the process into somewhat evenly long stages
- The more stages you have the more overhead you have from loading registers

■ Percentage of clock cycle spent loading register:	
● 1-stage pipeline:	6.25%
● 3-stage pipeline:	16.67%
● 6-stage pipeline:	28.57%

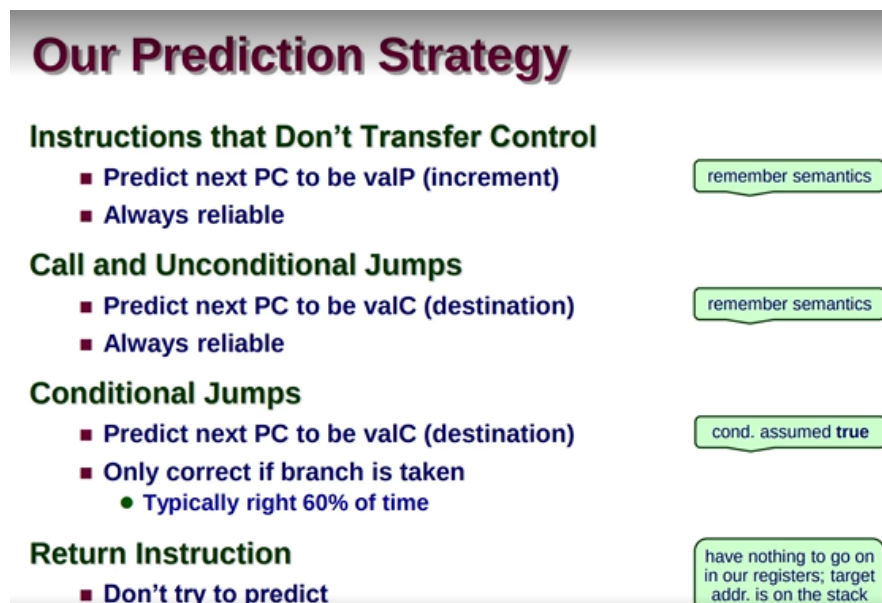
When is data pipelining a good idea?

- **When you don't have data dependencies**
 - For example, when an operation depends on the result of the previous operation
 - Using pipelining when you have dependencies can result in a "data hazard" - getting incorrect results because the appropriate dependencies aren't given (in time)
 - **The point of pipelining is: "same behaviour, just faster"**

Changes to adapt SEQ hardware (fully sequential) for pipelining

- **SEQ+ (smallest change)**
 - Still a sequential implementation, but reorder the PC stage at the beginning instead
 - PC no longer stored in registers
- **Adding pipeline registers**
 - Between each of the stages of the CPU we put a really long register
 - Pipeline stages:
 - Fetch (select current PC, read instruction, compute next PC)
 - Decode (read registers)
 - Execute

- Memory (read or write to memory)
- Write back (update register file)
- **PIPE-hardware**
 - Pipeline registers hold intermediate values from instruction execution
 - The paths are upward
 - Values passed from one stage to the next
 - Can't jump past stages
 - Also has feedback paths
 - Guess values of next PC
 - It guesses it based on what it would've been if the previous instruction was not a jump or return but just something like an add
 - Hopefully mostly guess correctly and recover if the prediction was wrong
 - Try to solve the data hazard problem



- Cost of misprediction: 2 jumps wasted if jump PC was guessed wrong and 3 jumps wasted if ret PC was guessed wrong

- (In practice it might actually flip the prediction on conditional jumps if it keeps getting it wrong - for example, if it predicts that the condition is true and it turns out it was wrong a few times, it will predict that the condition is false from then on)
- If you have dependencies, then you risk data hazards - one work-around you can use is to add `nop` instructions ("do nothing") so that the instruction before the `nop` has time to finish and the correct value is passed to the instruction which has the dependency

Pipeline Summary, so far

Concept

- Break instruction execution into 5 stages
- Run instructions through in pipelined mode

Limitations

- Can't handle dependencies between instructions when instructions follow too closely
- Data dependencies
 - One instruction writes register, later one reads it
- Control dependency
 - Instruction sets PC in way that pipeline did not predict correctly
 - Mispredicted branch and return

How to handle these? coming right up...

spoiler: dynamically
inject the right number of
`nop`s

Basic strategy for preventing these limitations:

Stalling

aka adding `nop`s dynamically

- If a "dangerous" situation is detected (usually if an instruction comes too closely after another instruction which writes a value to a register) slow it down by adding `nop`s
- A single `nop` injected is called a "bubble"
- Directly pass data from the instruction that generates it to the decode stage - "data forwarding"

- Always forward the most recent value (for example, if you perform 3 instructions on `rax` that change its value pass the value after the last instruction)
- This only works if you're working with register data and not with memory loaded data (until the data from memory was loaded into a register)

Detection

Condition	Trigger
Processing <code>ret</code>	<code>IRET</code> in { <code>D_icode</code> , <code>E_icode</code> , <code>M_icode</code> }
Load/Use Hazard	<code>E_icode</code> in { <code>IMRMOVQ</code> , <code>IPOPQ</code> } && <code>E_dstM</code> in { <code>d_srcA</code> , <code>d_srcB</code> }
Mispredicted Branch	<code>E_icode</code> = <code>IJXX</code> & <code>!e_Cnd</code>

Action (on next cycle)

Condition	F	D	E	M	W
Processing <code>ret</code>	stall	bubble	normal	normal	normal
Load/Use Hazard	stall	stall	bubble	normal	normal
Mispredicted Branch	normal	bubble	bubble	normal	normal

Performance metrics

- **Clock rate**
- **Cycles per instruction (CPI)** - on average how many clock cycles does each instruction require

CPI for PIPE

CPI ≈ 1.0

- Fetch instruction each clock cycle
- Effectively process new instruction almost every cycle
 - Although each individual instruction has latency of 5 cycles

CPI > 1.0

- Sometimes must stall or cancel branches

Computing CPI

- `C` clock cycles
- `I` instructions executed to completion
- `B` bubbles injected ($C = I + B$)

$$CPI = C/I = (I+B)/I = 1.0 + B/I$$
- Factor B/I represents average penalty due to bubbles

- CPI ~ 1 is the ideal situation

Vectorization

- Instructions which operate on multiple data items at once
- Is implemented as instruction set extensions - many registers are added
 - AVX-512 - newest - 32 new registers - zmm
 - AVX2 - what was available in prflab - 16 new registers - ymm - 256 bits in length
 - SSE - 16 new registers - xmm - 128 bit

